



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3

по курсу «Языки и методы программирования»

«Класс булевских матриц с порядком на основе подсчёта равных строк и столбцов»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Индивидуальный вариант	1
Реализация	1
Класс BoolMatrix	1
HTTP-сервер	4
Протокол тестирования	4
Тест 1: Матрица без равных строк и столбцов	5
Тест через веб-сервер	5
Домашнее задание	5
Архитектура решения	5
Веб-интерфейс	6
Вывод	6

Цель работы

Целью данной работы является изучение интерфейсов и классов коллекций в языке Java, реализация интерфейса Comparable для создания упорядочиваемых типов данных.

Индивидуальный вариант

Класс булевских матриц размера $m \times n$ с порядком на основе суммарного количества строк и столбцов, все элементы которых равны между собой.

Дополнительно выполнено:

- Добавление лабораторной работы в веб-сервер (+1 балл);
- Запуск на VDS (+1 балл).

Реализация

Создан класс BoolMatrix, реализующий интерфейс Comparable<BoolMatrix>.

Класс BoolMatrix

Класс BoolMatrix представляет булевскую матрицу размера $m \times n$ и содержит:

- Конструктор из `int[][]`;
- Конструктор из строки и размеров;
- Метод `getOrd()` для вычисления порядка (количество строк и столбцов, где все элементы равны между собой);
- Метод `compareTo()` для сравнения матриц по порядку;
- Метод `toString()` для строкового представления.

BoolMatrix.java (конструкторы)

```
public class BoolMatrix implements Comparable<BoolMatrix> {
    private boolean[][] matrix;
    private int m, n;

    public BoolMatrix(int[][] matrix) {
        m = matrix.length;
        if (m > 0) {
            n = matrix[0].length;
        } else {
            n = 0;
        }

        this.matrix = new boolean[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                this.matrix[i][j] = matrix[i][j] == 1;
            }
        }
    }

    public BoolMatrix(String data, int rows, int cols) {
        this.m = rows;
        this.n = cols;
        this.matrix = new boolean[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int idx = i * n + j;
                this.matrix[i][j] = data.charAt(idx) == '1';
            }
        }
    }
}
```

BoolMatrix.java (метод getOrd)

```
public int getOrd() {
    int total_good_rows_and_cols = 0;

    for (int i = 0; i < m; i++) {
        boolean is_good = true;
        for (int j = 1; j < n; j++) {
            if (matrix[i][j] != matrix[i][0]) {
                is_good = false;
                break;
            }
        }

        if (is_good) {
            total_good_rows_and_cols++;
        }
    }

    for (int j = 0; j < n; j++) {
        boolean is_good = true;
        for (int i = 1; i < m; i++) {
            if (matrix[i][j] != matrix[0][j]) {
                is_good = false;
                break;
            }
        }

        if (is_good) {
            total_good_rows_and_cols++;
        }
    }

    return total_good_rows_and_cols;
}
```

BoolMatrix.java (методы compareTo и toString)

```
public int compareTo(BoolMatrix other) {
    int ord = getOrd();
    int other_ord = other.getOrd();

    if (ord > other_ord) {
        return 1;
    } else if (ord < other_ord) {
        return -1;
    }
    return 0;
}

public String toString() {
    String result = "";

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j]) {
                result += "1";
            } else {
                result += "0";
            }
            if (j != n - 1) {
                result += " ";
            }
        }
        if (i != m - 1) {
            result += "\n";
        }
    }
    return result;
}
```

HTTP-сервер

Сервер реализован в файле `Server.java` с использованием встроенного класса `com.sun.net.httpserver.HttpServer`. Сервер:

- Слушает порт 7513;
- Обрабатывает GET-запросы;
- Принимает параметры двух матриц (`mat1`, `mat2`) с их размерами;
- Возвращает JSON-ответ с представлением матриц и их порядком.

Протокол тестирования

Запуск сервера производится командой:

server@net1.yss.su

```
java Server.java
```

Тестирование выполняется с помощью утилиты curl.

Тест 1: Матрица без равных строк и столбцов

arseny@local-pc

```
$ java Test.java
1 0 0 1
0 1 1 0
1 1 0 0
This matrix has 0 equal rows & cols

1 0 0 1
1 1 1 1
1 1 0 1
1 1 0 1
This matrix has 3 equal rows & cols

1 1
1 1
This matrix has 4 equal rows & cols
```

Тест через веб-сервер

arseny@local-pc

```
$ curl "http://net1.yss.su:7513/?arr_lenght=2&mat1_n=2&mat1_m=2&mat1=0101&mat2_n=2&mat2_m=2&mat2=1111"
[
{"matrix": "0 1\n0 1", "metadata": "This matrix has 2 equal rows & cols"},
{"matrix": "1 1\n1 1", "metadata": "This matrix has 4 equal rows & cols"}
]
```

Домашнее задание

В рамках домашнего задания было необходимо реализовать веб-сервер для учёта лабораторных работ и летучек.

Архитектура решения

Сервер реализован на Java с использованием встроенного HTTP-сервера com.sun.net.httpserver. Рендеринг HTML-страниц выполняется на стороне сервера

— при каждом запросе формируется полная HTML-страница, которая отправляется клиенту. Для стилизации используется фреймворк Pico.css, подключаемый через CDN.

Сервер поддерживает:

- Отображение списка лабораторных работ и летучек;
- Добавление новых записей;
- Удаление записей;
- Хранение данных в памяти (во время работы сервера).

Веб-интерфейс

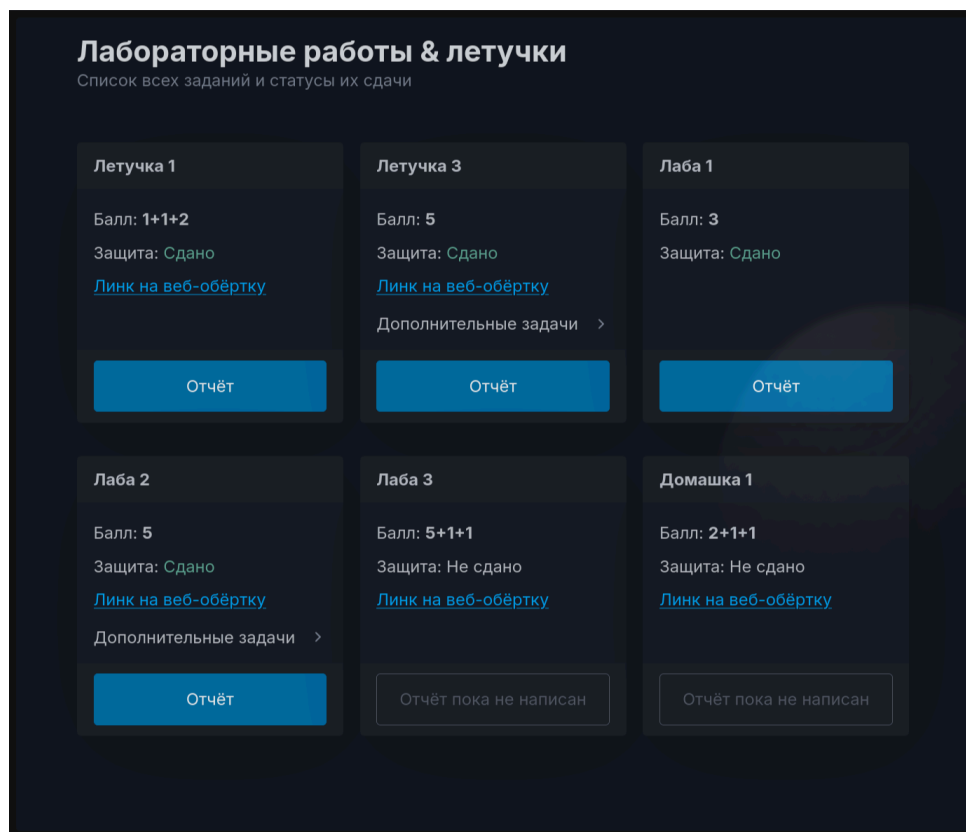


Рис. 1. Веб-интерфейс системы учёта лабораторных работ и летучек

Вывод

В ходе лабораторной работы был реализован класс `BoolMatrix`, представляющий булевские матрицы размера $m \times n$. Класс реализует интерфейс `Comparable<BoolMatrix>`, позволяющий сравнивать матрицы по порядку — суммарному количеству строк и столбцов, все элементы которых равны между собой.

Созданный HTTP-сервер успешно обрабатывает GET-запросы, парсит параметры матриц и возвращает результаты в формате JSON. Тестирование подтвердило корректность работы алгоритма вычисления порядка для различных матриц.